

Software Design Specification

Popup Core Programmers Guide

Version : 1.3
Issue Date : 5-oct-2013
Author : nkrishn9



**GUI,
Visteon Corporation**

INDEX

1	Introduction	3
2	Purpose	3
3	Definitions/Acronyms	3
4	Overview	4
5	Popup Classification	5
5.1	Info Popup	5
5.2	Key Pad Entry Popups	5
5.3	Progress bar Popups.....	6
5.4	Acknowledge Popup	6
6	Priority Management.....	7
7	Popup Layers	8
8	Queuing Management	9
9	Data structures	10
9.1	Data Type Definition	10
9.2	Data Generation	14
10	Integrating Popup Core.....	20
11	Popup Manager Public Interfaces	21
11.1	hmi_popup_core_layer<LAYER_NO>_rx_button_handler	21
11.2	hmi_popup_core_layer<LAYER_NO>_disp_foucus_handler.....	21
11.3	hmi_popup_core_layer<LAYER_NO>_disp_presentation_handler	21
11.4	hmi_popup_core_activate_popup	22
11.5	hmi_popup_core_deactivate_popup.....	22
11.6	hmi_popup_core_forced_deactivate_popup	22
11.7	hmi_popup_core_timeout_callback	22
11.8	hmi_popup_core_inspect_popup_active	23
11.9	hmi_popup_core_get_popup_info.....	23
11.10	hmi_popup_core_initialize	23
11.11	hmi_popup_core_start_timer	23
11.12	hmi_popup_core_stop_timer	24
11.13	hmi_popup_core_get_keypad_content	24
11.14	hmi_popup_core_set_keypad_content	24
11.15	hmi_popup_core_set_keypad_entry.....	24
11.16	hmi_popup_core_get_progress_bar_counter.....	24
11.17	hmi_popup_core_set_progress_bar_counter	24
11.18	hmi_popup_core_task.....	25
12	Revision history.....	26

1 Introduction

This document describes the popup core responsibilities, popup types, interaction with view manager, priority management, popup layer, popup queue, data structures and its data definition and popup core interfaces.

2 Purpose

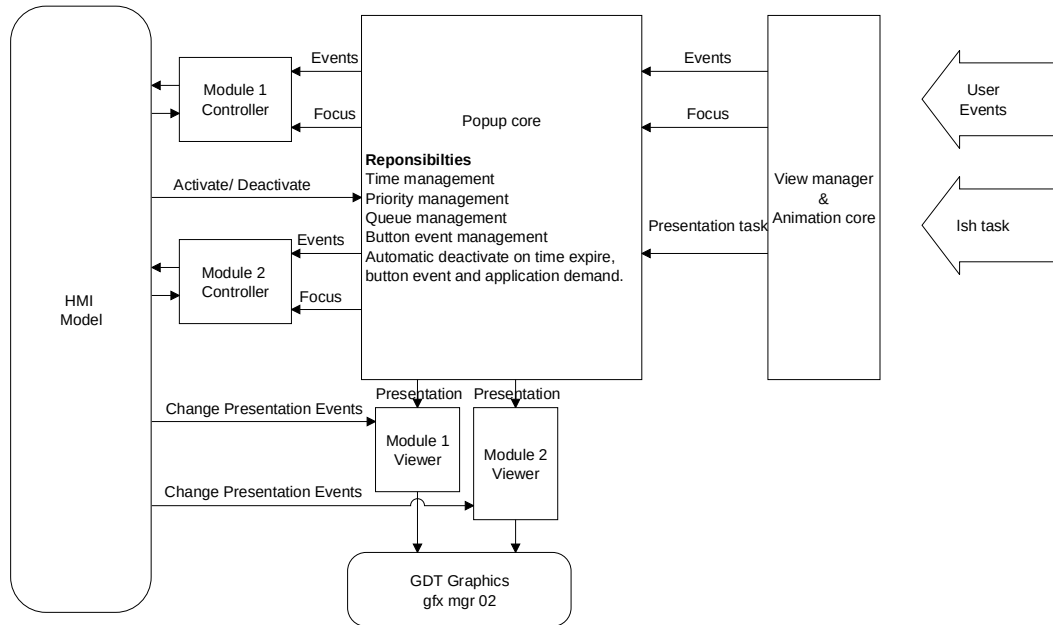
The purpose of the document is to understand the popup interfaces and its internal data organization, interfaces and responsibilities.

3 Definitions/Acronyms

Term	Definition
HMI	Human Machine Interface
HLD	High Level Design
DD	Detailed Design
TBD	To Be Defined
MC	Motion Control
LIFO	Last In First Out

4 Overview

Popup core is a base frame work for Popups of GUI ice apple architecture. It provides a single point of contact for various modules that want to display their own popup's. The Popup core makes the decision of displaying which popup on display. This brings down the problem of having this decision on each module to place holder. In below diagram Fig 4.1, the popup core interactions with different module controller and its viewer along with view manager and animation core is shown.



[Fig 4.1] Popup core Context with MVC architecture

Popup core is responsible for maintain the screen display time, prioritize the screen within a layer, same priority popup are grouped and queued as LIFO queue, button event to remove a screen, keypad data collection (like phone number, cd track entry etc), and progress bar management of screens. Here the popup logic state screen id is added to view manager by popup core whenever corresponding popup gains the highest priority in that popup layer. On added the highest priority popup logic state screen id, the view manager will update its event, focus and presentation handler of this popup layer which is passed on to the active popup event, focus and presentation handler which present in respective module controller and viewer as shown in above context diagram. The details of focus, presentation and event handler are explained in view manager.

5 Popup Classification

Popup has been categorized into four categories based on their functionality as follows,

- 1 Info Popup
- 2 Key Pad Entry Popup
- 3 Progress Bar Popup
- 4 Acknowledge Popup

5.1 Info Popup

The info popup are just the information popup that are put up in the display just to inform the user. Popup core does not handle any button processing on these popups. It is up to the application to handle the button events for this type of popup. The priority of popup and its screen id, timer to close the popup automatically after its elapse, minimum time display to avoid flickering issue, group priority for popups which share same priority, self-deactivate for the popup which configured as yes when high or equal priority popup activated, last in first display for group priority popup are the important characteristics of info popup.

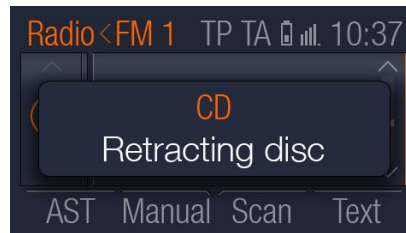


Fig [5.1] Info popup screen

5.2 Key Pad Entry Popups

Additional to info popup characteristics, keypad popup allows the user to enter the data through the standard 12 button key pad. Eg, CD tracks entry, entering a phone no. The entry is again categorized into,

1. Numeric
2. Alphabetic
3. Alphanumeric

The developer has to configure the respecting button events for the 12 button matrix in the configurator tool provided. The popup core process these button only if those are not processed by the application.

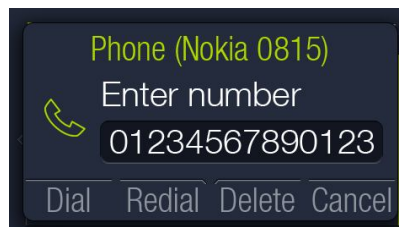


Fig [5.2] Keypad popup screen

5.3 Progress bar Popups

Additional to info popup characteristic, progress bar popup shows the status or the percentage complete or a time bar to the user. The developer has to configure the progress rate of popup in the configurator tool.

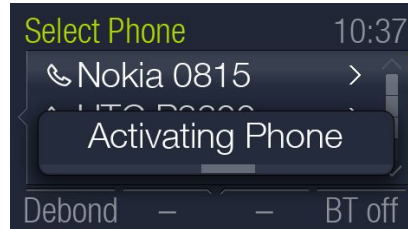


Fig [5.3] Progress bar popup screen

5.4 Acknowledge Popup

Additional to info popup characteristics, acknowledge popup displays information to the user as the information popup but waits for a user acknowledgement. The developer has to configure an acknowledge event in the configurator tool.

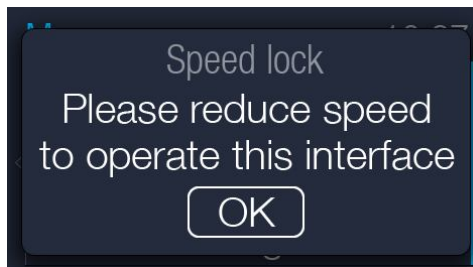


Fig [5.4] Acknowledge popup screen

6 Priority Management

The basic properties which are common for all popup types are defined in the POPUP_CONFIG itself as the order of the priority. Here, the first popup is the highest priority and the last is lowest one. So, each popup will have its priority as its order from top to down. The other properties which are specific for keypad, progress and acknowledge are defined in their respective structure. The index to get the properties of specific popup characteristics based on popup type is defined here. Ignore the index if the popup type is Info as all properties are defined in POPUP_CONFIG itself.

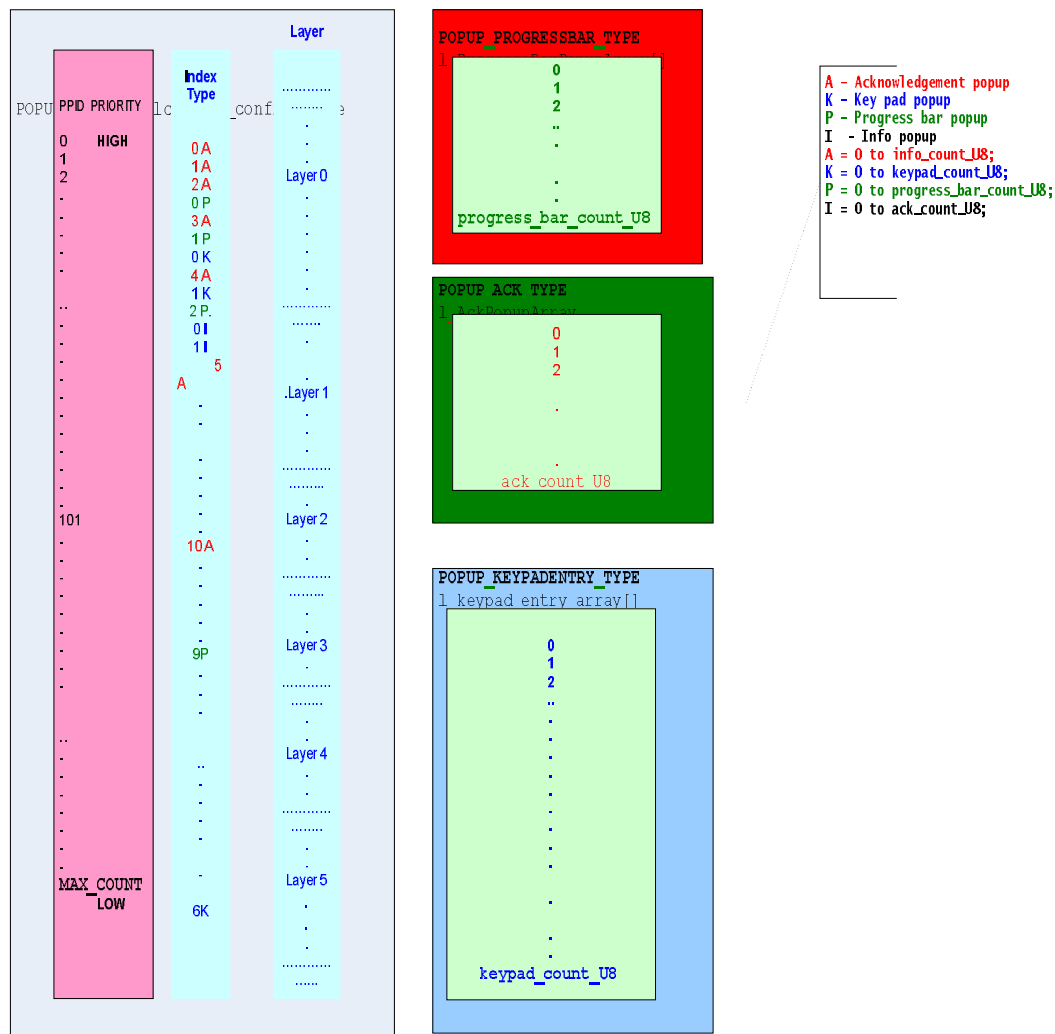


Fig [6.1] Popup Priority and Type abstract

7 Popup Layers

The popup id '0' defined is the highest priority popup which obviously falls in the layer '0' of popup core. The screen id of this popup in view manager is highest screen priority (of view manager) among the popup screen defined. All popup logic state screen id which shares this priority in view manager will fall under this layer 0. Similarly, each logic state screen id priority will form Layer 0, Layer 1 and up to Layer N as high to low priority layers of popup core. The focus, presentation and event handler of view manager which has same priority are mapped to the one popup layer event, focus and presentation handlers of popup core. Only one popup screen in each layer is active in view manager (since sharing same priority) at any point of time. On deactivate of this screen by popup core, the next priority popup logic state screen id will be added to view manager until all popups are displayed.



[Fig 7.1] Layer Decomposition Diagram

The properties which are common for a popup layer are defined in the `POPUP_LAYER_TYPE` structure. An index to find a layer table is defined in layer array structure. Each layer will have a list of group priority table and the group priority will have queue and its size is abstracted as shown above diagram. The total info, keypad, progress bar and acknowledgement popup are captured in a separate single instance structure.

8 Queuing Management

The popups which share a single priority in a layer are formed as group. Then, the first popup (from top to down as shown priority diagram) priority will be the priority of all the popup which belongs to that group. In the group, the popup which is added last (LIFO) will gain the highest priority of that group popup. The popup which are above this group priority are deactivated (i.e. when this group gained the priority in the layer), then the popup which is top of the group (i.e. the most recently added one) logic state screen id will be added to view manager. All the popup will be displayed as LIFO queue until group queue is emptied. Then, the popup core start continues to display one by one lower its priority as its order. SELF DEACTIVATE is a property of a popup which deactivate self (automatic by popup core) if any popup high or equal priority is added in that layer. So, the queue size of a group is number of popup with NO SELF DEACTIVATE plus one SELF DEACTIVATE. Since, the SELF DEACTIVATE popup automatically deactivate if above or equal priority is added in that layer.

The layer table contains the list of group priority table of that layer and the group priority will have its size and pointer to its size queue array.

9 Data structures

9.1 Data Type Definition

The popup type, group priority, timer type, self-deactivation, active popup deactivation disabled or not macros are defined as follows,

9.1.1 POPUP TYPES

```
#define POPUP_INFO      (0x01)
#define POPUP_KEYPAD    (0x02)
#define POPUP_PROGRESS  (0x03)
#define POPUP_ACK       (0x04)
#define LAST_POPUP_TYPE (POPUP_ACK)
```

9.1.2 GROUP PRIORITY

```
#define NO_GROUP_PRIORITY (0x00)
#define GROUP_PRIORITY_1  (0x01)
#define GROUP_PRIORITY_2  (0x02)
#define GROUP_PRIORITY_3  (0x03)
#define GROUP_PRIORITY_4  (0x04)
#define GROUP_PRIORITY_5  (0x05)
#define GROUP_PRIORITY_6  (0x06)
#define GROUP_PRIORITY_7  (0x07)
#define GROUP_PRIORITY_8  (0x08)
#define LAST_GROUP_PRIORITY (GROUP_PRIORITY_8)
```

9.1.3 TIMER TYPE

```
#define RESTART_TIMER    (0x00)
#define ALLOW_TO_EXPIRE  (0x02)
```

9.1.4 SELF DEACTIVATE

```
#define NO_SELF_DEACTIVATION (0x00)
#define SELF_DEACTIVATION   (0x01)
```

9.1.5 ACTIVE POPUP DEACTIVATION

```
#define DEACTIVATION_DISABLED (0x00)
#define DEACTIVATION_ENABLED  (0x01)
```

9.1.6 POPUP ID

```
typedef uint8_t POPUP_ID_T;
```

9.1.7 POPUP ATTRIBUTES

```
/* Bit 0-2 popup type
   001 INFO
   010 KEYPAD
   011 PROGRESS
   100 ACK

   Bit 3-6 Group priority
   0000 NO_GROUP_PRIORITY
   0001 GROUP_PRIORITY_1
   0010 GROUP_PRIORITY_2
   0011 GROUP_PRIORITY_3
   ...
   FFFF GROUP_PRIORITY_15
   Bit 7 reserved */
typedef uint8_t POPUP_ID_ATTRIBUTE_T;
```

9.1.8 POPUP INDEX

```
/* Popup type index
   the index will point to the popup type structure which will provide data of the popup id */
typedef uint8_t POPUP_ID_TYPEINDEX_T;
```

9.1.9 POPUP CONFIG

/ Bit 0-1 This is the type definition of the Timer type which will be considered when a popup is overridden by a popup of higher priority in the same layer*

Timer Type

00 RESTART_TIMER Restart the timer with timeout value when it is refocused

01 ALLOW_TO_EXPIRE Allow the timer to expire in the background

Bit 2-3 reserved

Bit 4 Deactivate the active popup in a layer

DEACTIVATE_ACTIVE_POPUP

Bit 5 All currently activated popup id with POPUP SELF DEACTIVATED config set in a group of a layer will get self deactivated if a same group priority popup activated

POPUP_SELF_DEACTIVATE

*Bit 6-7 reserved */*

```
typedef uint8_t POPUP_ID_CONFIG_T;
```

9.1.10 POPUP CONFIG

```
typedef struct
```

```
{
    POPUP_ID_ATTRIBUTE_T type;3; /*Bit 0-2 */
    POPUP_ID_ATTRIBUTE_T group_priority;4;
    /*POPUP_ID_ATTRIBUTE_T type_reserved:1;*/ /* reserved */
    POPUP_ID_TYPEINDEX_T index;
    POPUP_ID_CONFIG_T timer;2;
    /*POPUP_ID_CONFIG_T timer_reserved:2;*/
    POPUP_ID_CONFIG_T self_deactivate_pp;1;
    POPUP_ID_CONFIG_T deactivate_active_pp;1;
    /*POPUP_ID_CONFIG_T config_reserved:2;*/
    LSH_STATE_ID_T lsh_id;
    uint16_t min_timeout;
    uint16_t timerid; /* Timer id will give layer timer id or allow to expire timer id's if not restart timer */
    uint16_t layer_timeout; /* Timeout value in ms */
    LSH_EVENT_HANDLER_T * pp_evt_handler_ptr;
    LSH_FOCUS_HANDLER_T * pp_focus_handler_ptr;
    LSH_PRESENT_HANDLER_T * pp_present_handler_ptr;
    POPUP_EVENT_HANDLER_T * pp_popup_handler_ptr;
}POPUP_CONFIG_T;
```

9.1.11 KEYPAD POPUP TYPE

```
typedef enum
```

```
{
    NUMERIC_ENTRY = 0,
    ALPHABET_ENTRY,
    ALPHA_NUMERIC_ENTRY,
    ENTRY_INVALID
}KEY_PAD_ENTRY_TYPE;
```

```
typedef struct
```

```
{
    uint16_t l_btn_event_U16;
    uint8_t l_char_count_U8;
    const HMI_CHAR * l_char_array;
}KEYPAD_EVENT_TYPE;
```

```
typedef struct
```

```
{
    uint8_t l_btn_count_U8;
    const KEYPAD_EVENT_TYPE * l_keypad_evt_ptr;
}KEYPAD_CONFIG_TYPE;
```

```
typedef struct
```

```
{
    uint8_t l_btn_count_U8;
    const uint16_t * l_btn_keypad_evt_U16;
}PP_BTN_EVENT_TYPE;
```

typedef struct

```

{
    KEY_PAD_ENTRY_TYPE pp_entry_type;          /* Key Pad Type Numeric\ Alphabet, Alphanumeric */
    KEYPAD_CONFIG_TYPE const * config_type;     /* Key Code Pad button Mapping Config */
    UINT8 l_max_char_U8;                       /* Maximum no of character */
    UINT16 l_multi_press_timeout;              /* Multi press timer value */
    const PP_BTN_EVENT_TYPE *l_ok_event;       /* Event that is considered as the acceptance of the entered value */
    const PP_BTN_EVENT_TYPE *l_delete_event;    /* Event that deletes the last entered character */
    const PP_BTN_EVENT_TYPE *l_clear_event;     /* Event that clear the all the entry */
    const PP_BTN_EVENT_TYPE *l_cancel_event;    /* Event that cancels the Entry */
    POPUP_VALIDATE_HANDLER_T * pp_validation_handler_ptr; /* Handler Function for the validation Handler */
    HMI_CHAR * l_entry_ptr;                    /* Pointer to var that hold the entered character */
    UINT16 * l_multi_timer_counter;            /* Pointer to hold the current multi press counter value */
    UINT16 * l_multi_press_counter;            /* Pointer to hold current pressed button to find multi press or not */
    UINT16 * l_curr_multi_press_btn;           /* Pointer to hold current pressed button to find multi press or not */
} POPUP_KEYPADENTRY_TYPE;

```

9.1.12 PROGRESS BAR POPUP TYPE

/ From the progress bar popup configuration time(Popup Timeout) and Progress Bar Rate timeout, max count will be calculated and available in auto generated code.
i.e.l_progress_max_count_U8=Popup Timeout/Progress Bar Rate timeout in .dat file
If no layer time configured(i.e. "popup time out" is zero), the counter should keep on increase, until application deactivate it */*

typedef struct

```

{
    UINT8 l_progress_max_count_U8;             /* l_progress_max_count_U8=Popup Timeout/Progress Bar Rate timeout */
    UINT8 *l_progress_counter_U8;              /* Counter to count the increment */
} POPUP_PROGRESSBAR_TYPE;

```

9.1.13 ACKNOWLEDGEMENT POPUP TYPE**typedef struct**

```

{
    UINT16 l_btn_event_U16;
    UINT8 l_btn_type_U8;
} ACK_EVENT_TYPE;

```

typedef struct

```

{
    UINT8 l_btn_count_U8;
    const ACK_EVENT_TYPE *l_btn_ack_evt_U16;
} ACK_CONFIG_TYPE;

```

typedef struct

```

{
    UINT8 l_btn_count_U8;
    const UINT16 *l_btn_nack_evt_U16;
} NACK_CONFIG_TYPE;

```

typedef enum

```

{
    PASSED,
    ABSORBED
} ACK_BTN_TYPE;

```

typedef struct

```

{
    const ACK_CONFIG_TYPE *ack_btn_config;     /* Configuration button ev of ack popup */
    ACK_BTN_TYPE l_nack_btn_type;              /* Passed or Absorbed type for nack */
    const NACK_CONFIG_TYPE *l_nack_btn_config; /* Configuration button ev of nack popup */
} POPUP_ACK_TYPE;

```

9.1.14 TOTAL POPUP DATA

```
typedef struct
{
    UINT16 info_count_U8;
    UINT16 keypad_count_U8;
    UINT16 progress_bar_count_U8;
    UINT16 ack_count_U8;
    POPUP_ACK_TYPE const * l_ack_pp_table_ptr;
    POPUP_PROGRESSBAR_TYPE const * l_progress_bar_pp_table_ptr;
    POPUP_KEYPADENTRY_TYPE const * l_keypad_entry_pp_table_ptr;
    POPUP_NOTIFY_CB_POPUP_STATE_HANDLER_T * pp_state_handler_ptr;

}POPUP_TOTAL_POPUPTYPE_COUNT_T;
```

9.1.15 POPUP LAYERS

/ L_LAYER0_GROUP_PRIORITY_COUNT is number of grouped priorities count in layer 0
Queue size for each grouped priorities as follows,
no of "No" + 1 (for all YES) in self deactivate pp id configuration for each group priority */*

```
typedef struct
{
    POPUP_ID_T * queue;
    const POPUP_ID_T queue_size;
}LAYER_GROUP_PRIORITY_TYPE;

/* Type definition to hold the Popup Layer information
** logic_state_priority
** l_layer_index
** l_min_tmr_id          - minimum timer for a layer
** l_pp_start_index
** l_total_pp_count
** l_total_grp_priority_count
** l_total_pp_count_U8   - Total no of Popups
** l_group_priority_table_ptr
** */
typedef struct
{
    UINT8 logic_state_priority;
    UINT8 l_layer_index;

    TIMER_ID_TYPE l_min_tmr_id;

    POPUP_ID_T l_pp_start_index;
    POPUP_ID_T l_total_pp_count;
    UINT8 l_total_grp_priority_count;
    LAYER_GROUP_PRIORITY_TYPE const ** const l_group_priority_table_ptr;

}POPUP_LAYER_TYPE;
```

9.2 Data Generation

9.2.1 POPUP CONFIGURATION DATA

```
static const POPUP_CONFIG_T lc_popup_config_table[L_POPUP_MAX_COUNT] =
{
    /*POPUP_INFO_0_0*/
    {
        POPUP_INFO           ,      /*Popup Type           */
        NO_GROUP_PRIORITY    ,      /*Popup Group          */
        0                    ,      /*Type Index           */
        RESTART_TIMER        ,      /*Timer Type           */
        NO_SELF_DEACTIVATION ,      /*Self Deactivate Option*/
        DEACTIVATION_ENABLED ,      /*Interruption Option  */
        GFX_SCR_00           ,      /*Screen Id            */
        20                   ,      /*Minimum Time Out     */
        GUI_POPUP_LAYER0_TMR_ID ,      /*Timer Id             */
        200                  ,      /*Popup Timer Out       */
        hmi_popup_info_0_0_event_handler ,      /*Event Handler        */
        hmi_popup_info_0_0_focus_handler ,      /*Focus Handler         */
        hmi_popup_info_0_0_presentation_handler ,      /*Presentation Handler  */
        hmi_popup_info_0_0_popupevent_handler ,      /*Popup Core Handler   */
    },
    /*POPUP_KEYPAD_0_1*/
    {
        POPUP_KEYPAD         ,      /*PopupType            */
        .
        .
    },
    /*POPUP_ACK_1_0*/
    {
        POPUP_ACK            ,      /*PopupType            */
        .
        .
    },
    /*POPUP_PROG_1_1 */
    {
        POPUP_PROGRESS       ,      /*PopupType            */
        .
        .
    },
    .
    .
};
```

9.2.2 KEYPAD DATA

```

/*****
General Key Pad Entry Config
*****/
static const HMI_CHAR l_TestConfig_ButtonID_1_char[3] = {((HMI_CHAR)'a'), ((HMI_CHAR)'b'), ((HMI_CHAR)'c')};

static const KEYPAD_EVENT_TYPE l_TestConfig_keyevent[] =
{
    { ButtonID_1, 3 ,&l_TestConfig_ButtonID_1_char[0]    }
};

static const HMI_CHAR l_Alphanumeric_ButtonID_1_char[1] = {((HMI_CHAR)'1')};
static const HMI_CHAR l_Alphanumeric_ButtonID_2_char[4] = {((HMI_CHAR)'a'), ((HMI_CHAR)'b'), ((HMI_CHAR)'c'),
((HMI_CHAR)'2')};

static const KEYPAD_EVENT_TYPE l_Alphanumeric_keyevent[] =
{
    {ButtonID_1, 1 ,&l_Alphanumeric_ButtonID_1_char[0]    },
    {ButtonID_2, 4 ,&l_Alphanumeric_ButtonID_2_char[0]    },
};

static const HMI_CHAR l_Numeric_ButtonID_1_char[1] = {((HMI_CHAR)'1')};
static const HMI_CHAR l_Numeric_ButtonID_2_char[1] = {((HMI_CHAR)'2')};
```

```

static const KEYPAD_EVENT_TYPE l_Numeric_keyevent[] =
{
    {ButtonID_1, 1, &l_Numeric_ButtonID_1_char[0]},
    {ButtonID_2, 1, &l_Numeric_ButtonID_2_char[0]},
};

static const KEYPAD_CONFIG_TYPE l_TestConfig_keyconfig = {1, l_TestConfig_keyevent};
static const KEYPAD_CONFIG_TYPE l_Alphanumeric_keyconfig = {12, l_Alphanumeric_keyevent};
static const KEYPAD_CONFIG_TYPE l_Numeric_keyconfig = {12, l_Numeric_keyevent};

static const UINT16 l_ok_event_1[] =
{
    ButtonID_K, ButtonID_K1, ButtonID_K2, ButtonID_ok3
};
.
.

static const PP_BTN_EVENT_TYPE l_ok_event[] =
{
    {4, l_ok_event_1}, {1, l_ok_event_2}, {2, l_ok_event_3}
};

```

Similarly delete, cancel and clear button event.

```

static HMI_CHAR l_POPUP_K_0_entry_data[6];
static UINT16 l_POPUP_K_0_timer_counter;
static UINT16 l_POPUP_K_0_press_counter;
static UINT16 l_POPUP_K_0_press_btn;

static const POPUP_KEYPADENTRY_TYPE l_keypad_entry_array[] =
{
    /*POPUP_KEYPAD_1_1*/
    {
        ALPHABET_ENTRY, /*KeypadEntryType */
        &l_Alphanumeric_keyconfig, /*Keypad Config */
        6, /*MaxChar */
        200, /*MTimeout */
        &l_ok_event[0],
        &l_delete_event[0],
        &l_clear_event[0],
        &l_cancel_event[0],
        hmi_popup_validation_hndler, /*ValidationHandler */

        l_POPUP_K_0_entry_data,
        &l_POPUP_K_0_timer_counter,
        &l_POPUP_K_0_press_counter,
        &l_POPUP_K_0_press_btn
    },
    ...
};

```

9.2.3 ACKNOWLEDGEMENT POPUP DATA

```
static const ACK_EVENT_TYPE l_TestAckConfig_event[3]=
{
  { ButtonID_1, ABSORBED },
  { ButtonID_2, ABSORBED },
  { ButtonID_3, PASSED }
};

static const UINT16 l_TestNackConfig_event[5]=
{
  ButtonID_4,
  ButtonID_5,
  ButtonID_6,
  ButtonID_7,
  ButtonID_8
};

static const ACK_CONFIG_TYPE TestAckConfig = {3, l_TestAckConfig_event };
static const NACK_CONFIG_TYPE TestNackConfig = {5, l_TestNackConfig_event };

static const POPUP_ACK_TYPE l_AckPopupArray[] =
{
  /*POPUP_ACK_1_3*/
  {
    &TestAckConfig,/*ACK Button Config          */
    PASSED,/*Not ACK Button Type                */
    &TestNackConfig/*Not ACK Button Config      */
  },
  ...
};
```

9.2.4 PROGRESS BAR DATA

```
/******
Progress Bar Counter Definitions
******/

static UINT8 l_POPUP_P_0_counter_U8;
static UINT8 l_POPUP_P_1_counter_U8;

static const POPUP_PROGRESSBAR_TYPE l_ProgressBarPopupArray[] =
{
  /*POPUP_PROG_1_0*/
  {
    5,/*popup timeout/progress bar rate timer with round off value,if no popup timeout(i.e. 0), then this counter value will be 255*/
    &l_POPUP_P_0_counter_U8
  },
  /*POPUP_PROG_5_2*/
  {
    255,/*popup timeout/progress bar rate timer with round off value,if no popup timeout(i.e. 0), then this counter value will be 255*/
    &l_POPUP_P_1_counter_U8
  },
};
```

9.2.5 GROUP POPUP

```
/* Macros for the No of Groups available in each Layer */

#define POPUP_Layer1_NUM_GROUPS (2)
#define POPUP_Layer5_NUM_GROUPS (2)

/* Q declaration for the Group Priority Management */

static UINT8 l_Layer1_group_priority_1_Q [2];
static UINT8 l_Layer5_group_priority_1_Q [2];
static UINT8 l_Layer5_group_priority_2_Q [2];

static const LAYER_GROUP_PRIORITY_TYPE l_Layer1_group_priority_1_struct =
{
    &l_Layer1_group_priority_1_Q[0],
    2
};

static const LAYER_GROUP_PRIORITY_TYPE l_Layer1_group_priority_2_struct =
{
    &l_Layer1_group_priority_2_Q[0],
    1
};
static const LAYER_GROUP_PRIORITY_TYPE l_Layer5_group_priority_1_struct =
{
    &l_Layer5_group_priority_1_Q[0],
    2
};
static const LAYER_GROUP_PRIORITY_TYPE l_Layer5_group_priority_2_struct =
{
    &l_Layer5_group_priority_2_Q[0],
    2
};
static const LAYER_GROUP_PRIORITY_TYPE l_Layer10_group_priority_1_struct =
{
    &l_Layer10_group_priority_1_Q[0],
    1
};

static const LAYER_GROUP_PRIORITY_TYPE * l_Layer1_group_priority_table[] =
{
    &l_Layer1_group_priority_1_struct,
    &l_Layer1_group_priority_2_struct,
};

static const LAYER_GROUP_PRIORITY_TYPE * l_Layer5_group_priority_table[] =
{
    &l_Layer5_group_priority_1_struct,
    &l_Layer5_group_priority_2_struct,
};

static const LAYER_GROUP_PRIORITY_TYPE * l_Layer10_group_priority_table[] =
{
    &l_Layer10_group_priority_1_struct,
};
```

9.2.6 LAYER DATA

```

static UINT8 l_layer0_min_timer_sts_U8[(3+ 7)/8]; /* byte Array to store the Minimum Time Out Timer Status for each Popups */

static const POPUP_LAYER_TYPE l_popup_layer0 =
{
    0, /* Logic State Priority */
    L_POPUP_LAYER0_ID, /* Layer ID */
    GUL_POPUP_LAYER0_MIN_TMR_ID, /* Minimum Timeout Timer */
    0, /* Start Index */
    3, /* Total Popup Count */
    POPUP_Layer0_NUM_GROUPS, /* Total Group Prio Count */
    (LAYER_GROUP_PRIORITY_TYPE const **) NULL
};

static UINT8 l_layer1_min_timer_sts_U8[(5+ 7)/8]; /* byte Array to store the Minimum Time Out Timer Status for each Popups */

static const POPUP_LAYER_TYPE l_popup_layer1 =
{
    1, /* Logic State Priority */
    L_POPUP_LAYER1_ID, /* Layer ID */
    GUL_POPUP_LAYER1_MIN_TMR_ID, /* Minimum Timeout Timer */
    3, /* Start Index */
    5, /* Total Popup Count */
    POPUP_Layer1_NUM_GROUPS, /* Total Group Prio Count */
    &l_Layer1_group_priority_table[0]
};

static const POPUP_LAYER_TYPE l_popup_layer5 =
{
    2, /* Logic State Priority */
    L_POPUP_LAYER5_ID, /* Layer ID */
    POPUP_MAX_TIMER, /* Minimum Timeout Timer */
    8, /* Start Index */
    5, /* Total Popup Count */
    POPUP_Layer5_NUM_GROUPS, /* Total Group Prio Count */
    &l_Layer5_group_priority_table[0]
};

static UINT8 l_layer9_min_timer_sts_U8[(1+ 7)/8]; /* byte Array to store the Minimum Time Out Timer Status for each Popups */

static const POPUP_LAYER_TYPE l_popup_layer9 =
{
    3, /* Logic State Priority */
    L_POPUP_LAYER9_ID, /* Layer ID */
    GUL_POPUP_LAYER9_MIN_TMR_ID, /* Minimum Timeout Timer */
    13, /* Start Index */
    1, /* Total Popup Count */
    POPUP_Layer9_NUM_GROUPS, /* Total Group Prio Count */
    (LAYER_GROUP_PRIORITY_TYPE const **) NULL
};

static const POPUP_LAYER_TYPE l_popup_layer10 =
{
    4, /* Logic State Priority */
    L_POPUP_LAYER10_ID, /* Layer ID */
    POPUP_MAX_TIMER, /* Minimum Timeout Timer */
    14, /* Start Index */
    4, /* Total Popup Count */
    POPUP_Layer10_NUM_GROUPS, /* Total Group Prio Count */
    &l_Layer10_group_priority_table[0]
};

static POPUP_LAYER_TYPE const * l_layer_table[POPUP_MAX_LAYERS] =
{
    &l_popup_layer0,
    &l_popup_layer1,
    &l_popup_layer5,
    &l_popup_layer9,
    &l_popup_layer10,
};

static const POPUP_TOTAL_POPUPTYPE_COUNT_T l_total_pp_type =
{
    9,
    4,
    2,

```

```
3,  
  &l_AckPopupArray[0],  
  &l_ProgressBarPopupArray[0],  
  &l_keypad_entry_array[0],  
  hmi_app_popup_core_update_handler  
};
```

10 Integrating Popup Core

1. Take a copy of the popup core configurator and archive it into the project specific folder
2. Follow the instruction in the popup core configurator for configuring the popup details.
3. Generate the code from the popup configurator
4. Archive it the project specific vob.
5. Define the handler functions such as the Button handler, focus handler, and presentation handler, keypad validation handler, popup state active and popup event handler.
6. Define the variables with the respective data type if you are using the Key pad entry popup or progress bar type.
7. Add these source file to your project compiler and liker list.
8. Invoke the popup core's initialization function before using the popup interface for proper functionality.
9. Call hmi_popup_core_task handler to receive all the focus status of popup deactivated by popup core by itself.

11 Popup Manager Public Interfaces

11.1 hmi_popup_core_layer<LAYER_NO>_rx_button_handler

Name:	<i>hmi_popup_core_layer0_rx_button_handler</i>
Arguments:	<i>LSH_EVENT_HANDLER_STATUS_T</i> <i>*p_event_hndl_status_SP</i>
Returns:	<i>UINT8</i>
Description:	<i>This function should be configured in the GDT's Event handlers for the popup in the <Layer No>. The View Manager will be invoking this function when ever there is a button press to be processed by the Popup Manager</i> <i>The <Layer No> should be replaced by the respective layer no.</i>

11.2 hmi_popup_core_layer<LAYER_NO>_disp_foucus_handler

Name:	<i>hmi_popup_core_layer0_disp_foucus_handler</i>
Arguments:	<i>LSH_FOCUS_HANDLER_STATUS_T</i> <i>*p_focus_hndl_status_SP</i>
Returns:	<i>None</i>
Description:	<i>This function should be configured in the GDT's focus handlers for the popup in the <Layer No>. The View Manager will be invoking this function when ever there is a Focus Status Change to be processed by the Popup Manager</i> <i>The <Layer No> should be replaced by the respective layer no.</i>

11.3 hmi_popup_core_layer<LAYER_NO>_disp_presentation_handler

Name:	<i>hmi_popup_core_layer0_disp_presentation_handler</i>
Arguments:	<i>LSH_PRESN_HANDLER_STATUS_T</i> <i>*p_presn_hndl_status_SP</i>
Returns:	<i>None</i>
Description:	<i>This function should be configured in the GDT's Presentation handlers for the popup in the <Layer No>. The View Manager will be invoking this function when ever there is a presentation update for a screen to be processed by the Popup Manager</i> <i>The <Layer No> should be replaced by the respective layer no.</i>

11.4 hmi_popup_core_activate_popup

Name:	hmi_popup_core_activate_popup
Arguments:	<i>POPUP_ID</i> p_popup_id, <i>POPUP_ANIMATION_ID</i> p_anim_id
Returns:	<i>UINT8</i>
Description:	This function should be called by the application with the Popup ID & Animation ID as parameters when ever it wants to activate a particular popup. The Application will be notified with the Activation Status from View Manager/Popup-Manger via return Status

11.5 hmi_popup_core_deactivate_popup

Name:	hmi_popup_core_deactivate_popup
Arguments:	<i>POPUP_ID</i> p_popup_id, <i>POPUP_ANIMATION_ID</i> p_anim_id
Returns:	<i>UINT8</i>
Description:	This function should be called by the application with the Popup ID & Animation ID as parameters when ever it wants to deactivate a particular popup. The Application will be notified with the deactivation Status from View Manager/Popup-Manger via return Status. The popup shall not be deactivated immediately if its minimum display timer has not elapsed.

11.6 hmi_popup_core_forced_deactivate_popup

Name:	hmi_popup_core_forced_deactivate_popup
Arguments:	<i>POPUP_ID</i> p_popup_id, <i>POPUP_ANIMATION_ID</i> p_anim_id
Returns:	<i>UINT8</i>
Description:	This function should be called by the application with the Popup ID & Animation ID as parameters when ever it wants to deactivate a particular popup. The Application will be notified with the deactivation Status from View Manager/Popup-Manger via return Status. The popup shall be immediately deactivated even if its minimum display timer has not elapsed.

11.7 hmi_popup_core_timeout_callback

Name:	hmi_popup_core_timeout_callback
Arguments:	<i>UINT16</i> p_timer_id <i>UINT16</i> p_event
Returns:	<i>None</i>
Description:	This function should be called by the applications timer module when a particular timer is expired. The timer module should pass the event that was registered during the start request of the same timer.

11.8 hmi_popup_core_inspect_popup_active

Name:	hmi_popup_core_inspect_popup_active
Arguments:	POPUP_ID p_popup_id, POPUP_CONTEXT_TYPE p_context_type
Returns:	BOOLEAN
Description:	This function should be called by the application with a popup ID and POPUP_CONTEXT_INPUT as parameter, This interface is used to know the whether the popup is active in the application input context or active in the display for the corresponding layer of the popup.

11.9 hmi_popup_core_get_popup_info

Name:	hmi_popup_core_get_popup_info
Arguments:	POPUP_INFO_DATA_TYPE * const p_info_data, POPUP_LAYER_ID p_layer_id
Returns:	None
Description:	This function should be called by the application with a popup layer ID and the structure to fill total count and active popup as parameter. This interface is used to know the total number of popups active in that popup layer and the active popup. The active popup is one which is present the display i.e. lsh display context.

11.10 hmi_popup_core_initialize

Name:	hmi_popup_core_initialize
Arguments:	none
Returns:	None
Description:	This function is called to initialize the popup core.

11.11 hmi_popup_core_start_timer

Name:	hmi_popup_core_start_timer
Arguments:	POPUP_ID p_popup_id
Returns:	BOOLEAN
Description:	This function should be called by the application to restart the timer when this popup is active in that layer.

11.12 hmi_popup_core_stop_timer

Name:	hmi_popup_core_stop_timer
Arguments:	POPUP_ID p_popup_id
Returns:	BOOLEAN
Description:	This function should be called by the application to disable the timer when the popup is active in that layer.

11.13 hmi_popup_core_get_keypad_content

Name:	hmi_popup_core_get_keypad_content
Arguments:	POPUP_KEYPAD_CONTENT_TYPE *const p_keypad_content
Returns:	None
Description:	This function will return the keypad content of that popup

11.14 hmi_popup_core_set_keypad_content

Name:	hmi_popup_core_set_keypad_content
Arguments:	POPUP_KEYPAD_CONTENT_TYPE *const p_keypad_content
Returns:	UINT8 length
Description:	This function used by application to set or initialize the content for a popup text of a keypad popup.

11.15 hmi_popup_core_set_keypad_entry

Name:	hmi_popup_core_set_keypad_entry
Arguments:	POPUP_KEYPAD_SET_TYPE const * const p_keypad_set
Returns:	None
Description:	This function used by application to set button events for a keypad popup.

11.16 hmi_popup_core_get_progress_bar_counter

Name:	hmi_popup_core_get_progress_bar_counter
Arguments:	POPUP_PROGRESS_COUNT_TYPE *p_progress
Returns:	BOOLEAN
Description:	Will return the progress counter for a popup

11.17 hmi_popup_core_set_progress_bar_counter

Name:	hmi_popup_core_get_progress_bar_counter
Arguments:	POPUP_PROGRESS_COUNT_TYPE *p_progress
Returns:	BOOLEAN
Description:	Will set the progress counter for a popup which affect the popup timer. The progress popup timer will be proportional to the the counter value set.

11.18 hmi_popup_core_task

Name:	hmi_popup_core_task
Arguments:	None
Returns:	None
Description:	To receive all the focus status for the popup deactivated by popup core automatically. The interface needs to be called after the lsh task function is called. It will process the popup which is deactivated by popup core without the knowledge of view manager and call back the popup focus handler of application to inform that the popup is deactivated.

12 Revision history

Version	Section	Date	Cdsid	Nature of change
1.0	All	22-Mar-13	Nkrishn9	Initial Version
1.1	7.4-7.6, 7.17, 71.18	24-Jun-13	Nkrishn9	Updated the popup task, POPUP_ANIMATION_ID and progress bar counter set.
1.2	5	1-SEP-2013	Nkrishn9	Added the respective characteristics of popup types.
1.3	All	5-OCT-2013	Nkrishn9	Emanoj comments to update priority management, popup context MVC diagram, queuing mechanism, data structures and its definition are added.